Activaciones Racionales Condicionadas por Contexto: Cuándo Ayuda Adaptar la No Linealidad

Agustín Hugo Astie Forconesi,Juan Pablo Gerino,Joaquin Imanol Ortega

Universidad del aconcagua,Universidad del aconcagua,Universidad del aconcagua
agustinastie3@gmail.com,juanpablo@gmail.com,Joaquin@gmail.com

Resumen Las redes neuronales tradicionales emplean funciones de activación estáticas (e.g., ReLU, SiLU) que imponen la misma transformación no lineal a todas las entradas. Esto representa una limitación cuando los datos provienen de escenarios diferentes (por ejemplo, mezclar imágenes normales con imágenes en negativo). En este trabajo proponemos la *Activación Racional Condicionada por Contexto* (CCA), A diferencia de las funciones fijas, CCA utiliza una pequeña red auxiliar para observar el contexto global de los datos y modificar la forma de su propia curva en tiempo real. A través de experimentos controlados en tareas de imágen y texto, demostramos que CCA supera consistentemente a los modelos base (con mejoras de entre 15 % y 46 %). Mediante pruebas de ablación causal, comprobamos que deformar la curva matemática es mucho más efectivo que la técnica tradicional de simplemente sumar el contexto a la entrada. Finalmente, demostramos un límite importante del método: si los datos son simples y pertenecen a un único régimen, la ventaja de usar parámetros dinámicos desaparece por completo.

Keywords: Deep Learning · Funciones de Activación Adaptativas · Hiperredes · Aprendizaje Condicionado.

1. Introducción

El diseño de funciones de activación es una pieza central en el desarrollo del Aprendizaje Profundo. Las funciones clásicas, como ReLU y SiLU, son estáticas: siempre aplican la misma curva sin importar qué datos estén procesando. No obstante, investigaciones recientes han demostrado que darle cierta flexibilidad a la activación mejora el aprendizaje. Por ejemplo, MishReLU [1] ajusta la pendiente en valores negativos, mientras que funciones como Dynamic ReLU [3], ASH [6] y ACON [7] adaptan su comportamiento según el canal de entrada. Además, se ha comprobado que las funciones racionales (basadas en fracciones de polinomios) son matemáticamente más eficientes y expresivas que las funciones tradicionales [2, 8].

A pesar de estos avances, surge un problema importante cuando un modelo debe procesar datos que provienen de múltiples regímenes incompatibles. Por ejemplo, si un conjunto de datos contiene imágenes normales mezcladas con

imágenes invertidas fotográficamente, lo que es un valor .ªlto"para una imagen, debería ser "bajo"para la otra. Una función de activación estática no puede hacer ambas cosas bien al mismo tiempo.

Para resolver este desafío, presentamos la Activación Racional Condicionada por Contexto (CCA). Utilizando el concepto de hiperredes [5, 9], CCA lee el contexto general de la entrada y lo utiliza para predecir cómo debe deformar su propia curva matemática. En este artículo, explicamos cómo funciona esta arquitectura y demostramos empíricamente cuándo resulta beneficioso adaptar la curva de activación en lugar de usar un enfoque estático.

2. Formulación de CCA

El diseño de CCA reemplaza la curva fija por una curva dinámica generada por el contexto. Para procesar una entrada x dado un contexto c, la operación se divide en dos pasos:

$$u = w(c) \cdot x + d(c) \tag{1}$$

$$A(x, c) = \underbrace{u \cdot \sigma(u)}_{\text{Base SiLU [4]}} + \alpha \underbrace{\frac{a_0 + a_1 u + a_2 u^2}{1 + (b_0 + b_1 u)^2}}_{\text{Modulación Racional}} \tag{2}$$

1. **Ajuste Inicial (Eq. 1):** Antes de aplicar la activación, los datos de entrada se escalan y desplazan usando coeficientes (w y d) dictados por el contexto. Esto permite que la red .ªlinee"los datos numéricamente antes de procesarlos.
2. **Base SiLU:** El primer término de la Ecuación 2 asegura que CCA herede la estabilidad de la función estándar Sigmoid Linear Unit [4].
3. **Deformación Racional Acotada:** La fracción polinomial es la encargada de deformar la curva. Es crucial notar que el denominador está elevado al cuadrado, más un 1. Esto asegura que la división nunca sea por cero y que los valores no .ˣploten"hacia el infinito, previniendo errores durante el entrenamiento.

Todos los parámetros matemáticos (w, d, a_n, b_n) se generan en el momento mediante una pequeña sub-red. Además, diseñamos una inicialización llamada Exact-SiLU. Al principio del entrenamiento, forzamos a que la fracción valga cero absoluto. Por lo tanto, la red arranca operando exactamente como un SiLU estático normal, y solo deforma su curva si descubre que hacerlo reduce verdaderamente el error.

3. Experimentos y Resultados

Evaluamos CCA en cuatro configuraciones de pequeño tamaño pero altamente controladas. En cada una, los datos consisten en múltiples regímenes subyacentes:

- **Generación de Texto:** El modelo debe predecir la siguiente letra, pero el texto está alterado por códigos de sustitución (por ejemplo, todas las 'A' cambian por 'M'). Para predecir bien, la red debe entender el contexto para descifrar la regla activa. Lo medimos usando Perplejidad, una métrica que indica qué tan confundido está el modelo (valores más bajos son mejores).
- **Generación de Imágenes:** Predicción de píxeles donde los colores de la imagen han sido invertidos o permutados. Se evaluó usando bits/dim, que mide la eficiencia de compresión de la imagen (valores menores significan mayor entendimiento estructural).
- **Clasificación de Imágenes:** Pequeña CNN sobre MNIST, sujeto a 8 transformaciones de píxeles (inversión, rotaciones, espejados).
- **Clasificación de Texto:** MLP basado en bag-of-embeddings sobre fuentes con ruido, midiendo directamente el Porcentaje de Error.

En las tareas de clasificación, todo el input es provisto a la red, probando directamente la capacidad de *computación condicional* de la capa. En las tareas de generación implementamos un *escenario estricto sin fugas de información* (no-leak setting), donde el contexto de CCA (e.g. historial pasado de 512 caracteres) está completamente separado y antecede a la ventana causal que ve la red (backbone).

3.1. Mejoras de Rendimiento

CCA logró ganancias y consistentes ($\geq 15\,\%$) en todas las tareas probadas, tal como se resume en la Tabla 1.

Cuadro 1. Mejora general en escenarios multi-régimen (Media de 5 semillas).

Tarea	**Métrica**	**Regímenes**	**Baseline (SiLU)**	**CCA**	**Mejora**
Gen. Texto	PPL	32	21.68 ± 0.42	**11.66** ± 0.25	-46.2 %
Clasif. Imagen	Error %	8	14.96 ± 0.99	**11.87** ± 0.72	-20.7 %
Gen. Imagen	bits/dim	16	1.029 ± 0.006	**0.872** ± 0.005	-15.2 %
Clasif. Texto	Error %	8	6.92 ± 2.48	**4.03** ± 1.51	-41.8 %

3.2. Control B0H: Sumar contexto vs Deformar la curva

Queríamos comprobar si la mejora de CCA se debía simplemente a que tenía acceso a más información del contexto. Para ello, creamos el control **Baseline B0H (Baseline + History)**. En B0H, usamos una activación estática común (SiLU), pero le sumamos la información del contexto directamente a la entrada, una técnica muy común en la literatura.

La Figura 1 demuestra que CCA es holgadamente superior a B0H. Esto prueba que alterar físicamente la forma de la curva de activación (el enfoque de CCA) es mucho más poderoso que simplemente inyectar la información del contexto sumándola a los datos.

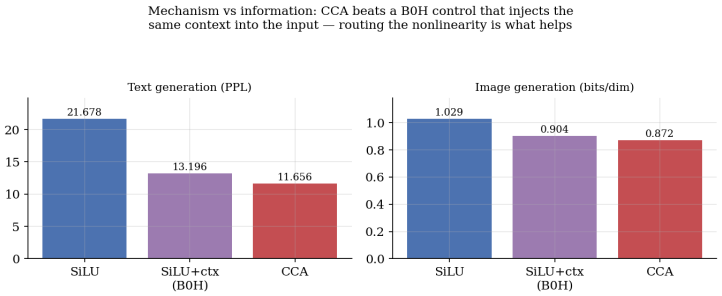


Figura 1. Figura 1. Reducción del error de CCA en comparación con sumar el contexto (B0H).

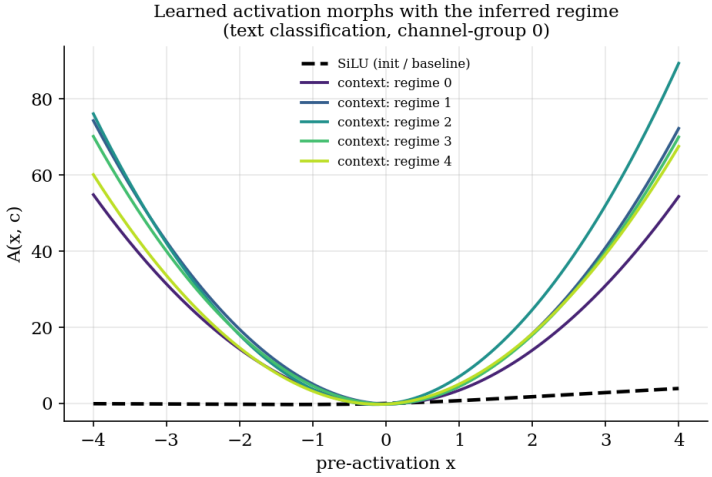


Figura 2. Figura 2.Ejemplos de cómo CCA modifica la forma estática SiLU para resolver diferentes regímenes.

3.3. Ablaciones y el Límite del Método

Para estar seguros de que el modelo estaba razonando y no solo memorizando, realizamos dos pruebas fundamentales de control:

- **Ablación de Barajado (Shuffle Ablation):** Decidimos engañar al modelo. Le dimos a predecir una imagen en negativo, pero le enviamos a la activación el contexto de una imagen normal. Ante esta trampa, el rendimiento de CCA se desplomó por debajo del baseline estático. Esto es algo positivo metodológicamente: demuestra que la red aprendió a depender estrictamente del contexto real para funcionar, confirmando una relación causal fuerte.
- **Límite Single-Regime:** Queríamos saber qué pasaba si los datos eran normales y fáciles. Limpiamos el dataset de cifrados e imágenes invertidas, dejando un único régimen simple. En este escenario, la enorme mejora de CCA se redujo casi al 0 %. El modelo se dio cuenta de que deformar la curva no aportaba ninguna ventaja y decidió seguir operando exactamente como un SiLU tradicional. Esto sirve como una valiosa advertencia científica: no se deben añadir activaciones dinámicas ni parámetros extra en problemas monótonos o simples.

4. Conclusión

En este artículo demostramos que las Activaciones Racionales Condicionadas por Contexto son altamente eficaces para resolver problemas donde los datos atraviesan diferentes regímenes (logrando mejoras superiores al 15 %). Demostramos empíricamente que permitirle a la red deformar su matemática interna en tiempo real es una estrategia muy superior a la inyección tradicional de contexto (control B0H). Al mismo tiempo, identificamos que este mecanismo es innecesario en bases de datos simples de un solo régimen. Entender claramente cuándo y por qué funcionan las activaciones dinámicas abre un camino prometedor para mejorar la eficiencia de futuros modelos fundacionales a gran escala.

Referencias

1. Al-nawi, N.: Mishrelu: A hybrid activation function for deep neural networks. No especificado (2025)
2. Boullé, N.: Rational neural networks. In: NeurIPS (2020)
3. Chen, Y., et al.: Dynamic relu. In: ECCV (2020)
4. Elfwing, S., Uchibe, E., Doya, K.: Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. In: CoRR (2017)
5. Ha, D.: Hypernetworks. In: ICLR (2017)
6. Lee, K., et al.: Adaptive swish (ash): adaptive activation function for deep neural networks. In: arXiv (2022)
7. Ma, N.: Activate or not: Learning customized activation. In: CVPR (2021)
8. Molina, A.: Padé activation units: Learning activation functions in deep neural networks. In: arXiv (2019)
9. Perez, E.: Film: Visual reasoning with a general conditioning layer. In: AAAI (2018)